

Tại Sao Chúng Ta Cần Giao Tiếp Thời Gian Thực?

Giới thiệu về Socket.IO trong Planner Web

So Sánh Mô Hình Giao Tiếp

HTTP Truyền Thống



Client-initiated, request-response.

- **Hạn chế:** Cần polling để cập nhật, overhead cao, không hiệu quả cho các tính năng tương tác tức thì.

Socket.IO Real-time



Bidirectional, event-driven.

- ✓ **Lợi ích:** Cập nhật tức thì, server có thể chủ động đẩy dữ liệu, kết nối bền vững.

Các Tính Năng Cốt Lõi Được Kích Hoạt Bởi Socket.IO



Nhắn tin (**chat**) thời gian thực



Thông báo hệ thống và lời mời kết bạn tức thì



Chỉnh sửa tài liệu cộng tác



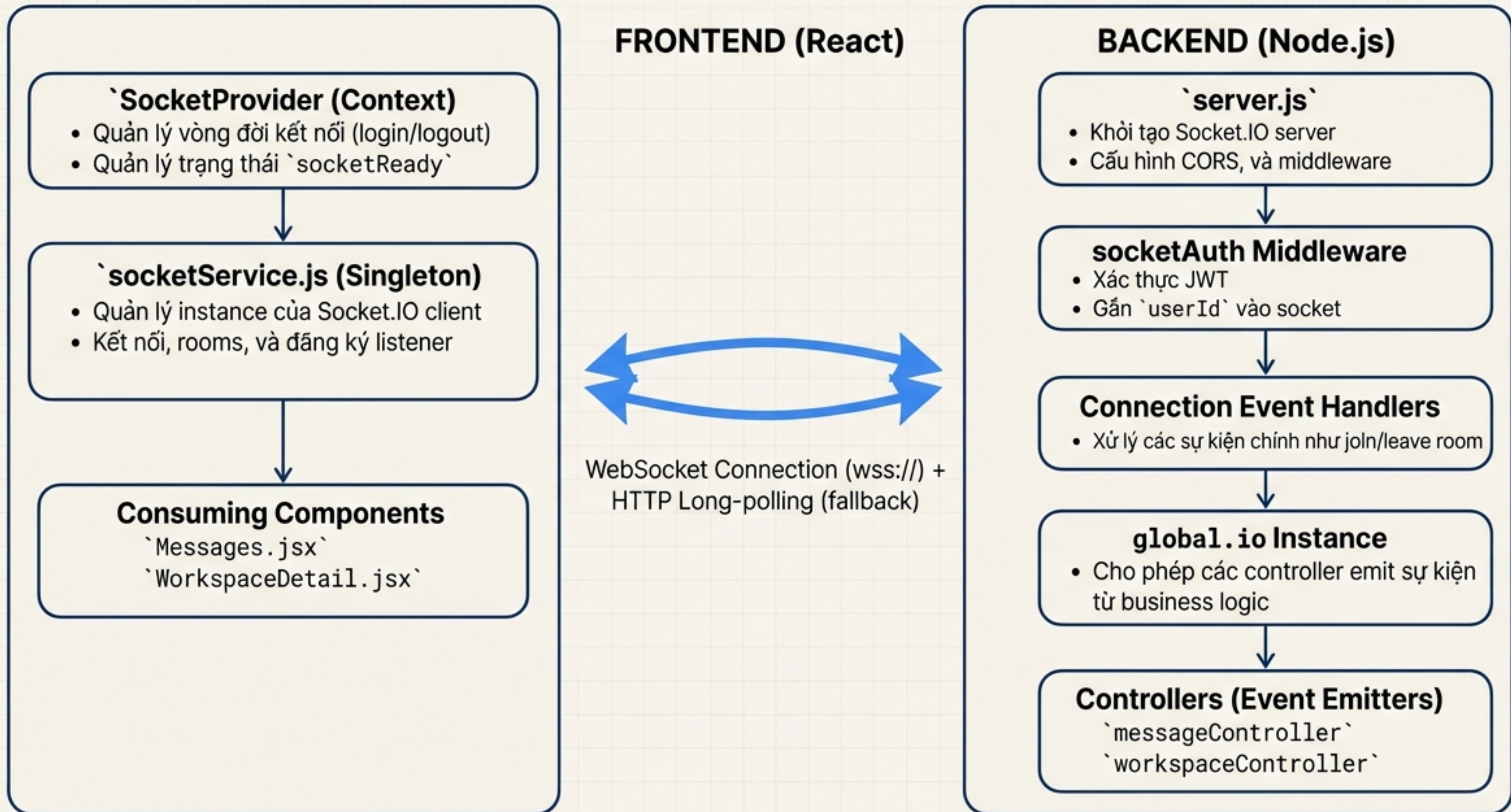
Cập nhật không gian làm việc trực tiếp



Đồng bộ hóa trạng thái công việc

Socket.IO không chỉ là WebSocket. Nó là một thư viện cấp cao cung cấp khả năng tự động kết nối lại, fallback qua HTTP polling, và một hệ thống sự kiện mạnh mẽ.

Sơ Đồ Kiến Trúc Toàn Cảnh



Nền Tảng: Thiết Lập và Bảo Mật Phía Backend

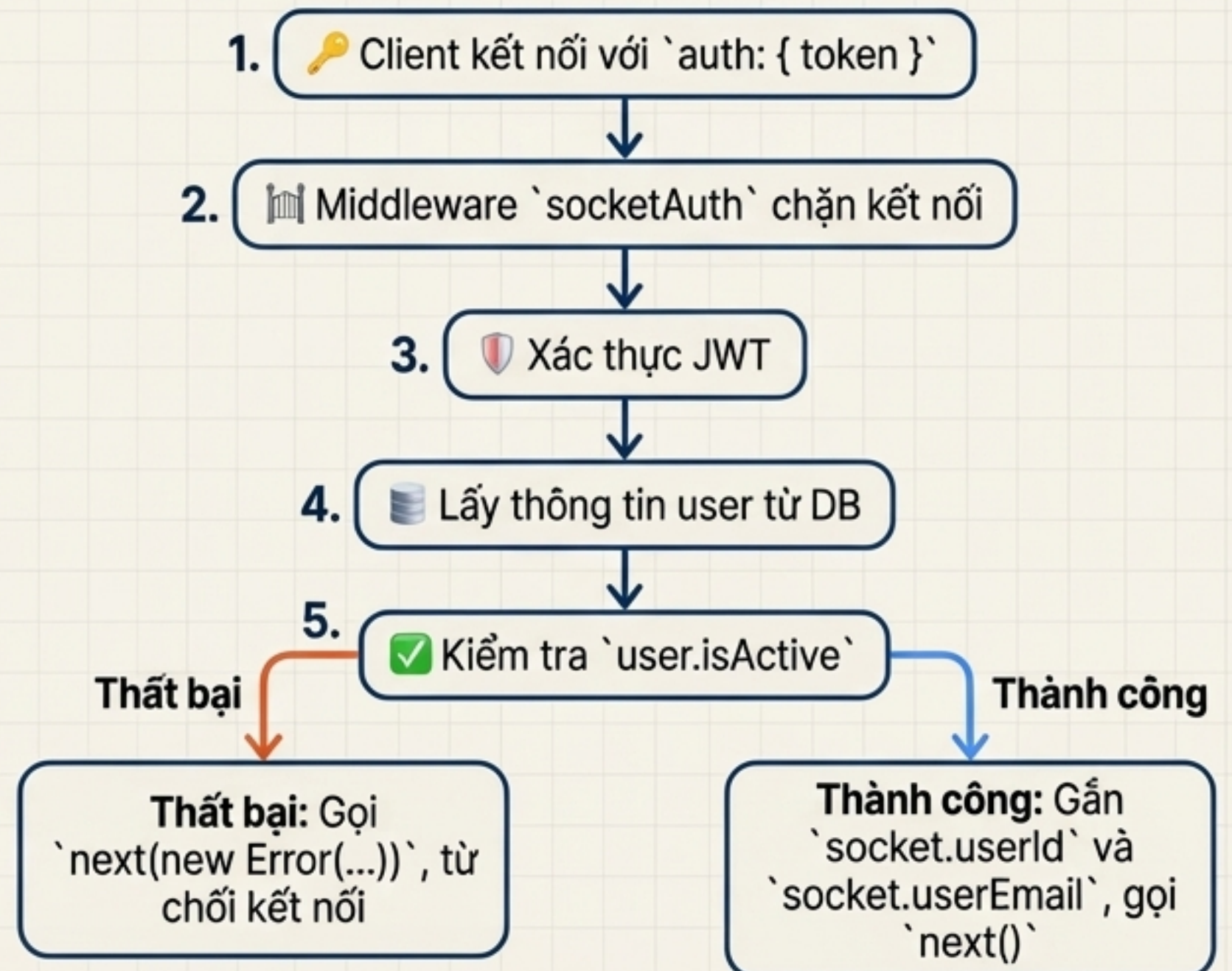
Khởi Tạo Server (`server.js`)

```
const io = require('socket.io')(server, {  
  cors: {  
    origin: config.cors.origin, // Frontend URL  
    methods: ["GET", "POST"],  
    credentials: true  
  }  
});
```

- ``cors.origin``: Đảm bảo chỉ frontend của chúng ta có thể kết nối.
- ``transports``: Ưu tiên 'websocket', fallback về 'polling'.
- ``reconnection``: Tự động kết nối lại được bật mặc định.

Cổng An Ninh - `socketAuth` Middleware

Mục đích: Xác thực mọi kết nối socket trước khi xử lý.



Lỗi Vận Hành: Xử Lý Kết Nối và Phát Sự Kiện

Xử Lý Sự Kiện `connection`

- **Hành vi chính:** Sau khi xác thực thành công, mỗi người dùng sẽ tự động tham gia vào một "phòng" cá nhân.

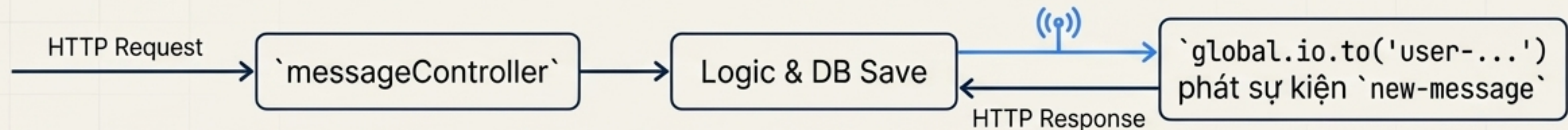
Thông tin có sẵn trên `socket`

- `socket.id`
- `socket.userId`
- `socket.userEmail`
- `socket.rooms`

```
io.on('connection', (socket) => {  
  console.log(`👤 User connected: ${socket.id} (userId: ${socket.userId})`);  
  
  // Tự động tham gia phòng cá nhân  
  const userRoom = `user-${socket.userId}`;  
  socket.join(userRoom);  
  
  // Đăng ký các trình xử lý sự kiện khác...  
  socket.on('disconnect', () => { /* ... */ });  
});
```

Pattern `global.io` - Phát Sự Kiện Từ Controllers

- **Vấn đề:** Làm thế nào để các controller (xử lý request HTTP) có thể gửi sự kiện socket?
- **Giải pháp:** Lưu instance `io` vào một biến global (`global.io`) khi khởi tạo server.



“HTTP API là nguồn chân lý (source of truth). Sự kiện Socket là một cơ chế tối ưu hóa để đồng bộ client trong thời gian thực.”

Trải Nghiệm Người Dùng: Quản Lý Kết Nối Phía Frontend

`SocketService` - Mô Hình Singleton

Mục đích: Tập trung toàn bộ logic quản lý socket vào một nơi duy nhất, đảm bảo chỉ có một instance kết nối tồn tại.

2. Khởi Tạo Kết Nối An Toàn

connect(userId, token)

- 1. Kiểm tra và ngăn chặn việc tạo kết nối trùng lặp.
- 2. Khởi tạo `io()` với `auth: { token }` để gửi JWT lên server.
- 3. Cấu hình các tùy chọn quan trọng: `transports`, `reconnection`, `reconnectionDelay`, `reconnectionAttempts`.
- 4. Đăng ký các trình xử lý vòng đời kết nối: `connect`, `disconnect`, `connect\_error`.

SocketService

Properties

-  socket (Instance IO)
-  currentUserId (String)
-  registeredListeners (Map)
(Event -> Callback[])

Methods

-  connect()
-  disconnect()
-  joinWorkspace()
-  _registerListener()

```
this.socket = io(SOCKET_URL, {  
  auth: { token }, // Gửi JWT để xác thực  
  reconnection: true,  
  reconnectionAttempts: 5  
});
```


Tích Hợp Giao Diện: Context và Quản Lý Listener An Toàn



`SocketContext` - Cung Cấp Toàn Cục

Mục đích: Cung cấp instance `socketService` và trạng thái `socketReady` cho toàn bộ cây component.

Logic trong `useEffect`

- Khi người dùng đăng nhập (`user` tồn tại), gọi `socketService.connect()`.
- Khi kết nối thành công (`connect` event), set `socketReady` thành `true`.
- Khi người dùng đăng xuất, gọi `socketService.disconnect()`.

Sử dụng trong Component

```
const { socketService, socketReady } = useSocket();
```



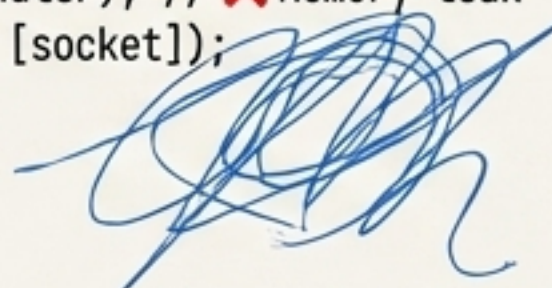
Chống Memory Leak - Quản Lý Listener Thông Minh

Vấn đề: Các component React mount/unmount liên tục. Nếu không dọn dẹp listener, chúng sẽ bị tích tụ và gây ra lỗi.

`SocketService` sử dụng một `Map` (`registeredListeners`) để theo dõi các listener đã được đăng ký.

❌ Bad (Không dọn dẹp)

```
useEffect(() => {  
  socket.on('new-message',  
    handler); // ❌ Memory leak  
}, [socket]);
```



✅ Good (Có dọn dẹp)

```
useEffect(() => {  
  socket.on('new-message',  
    handler);  
  return () => {  
    socket.off('new-message',  
      handler); // ✅ Dọn dẹp khi unmount  
  };  
}, [socket]);
```

⚠️ **Lưu ý:** `SocketService` cung cấp các phương thức `\_registerListener` và `\_unregisterListener` để tự động hóa việc này và tránh đăng ký trùng lặp.

Cơ Chế Cốt Lõi: Hệ Thống Room và Các Mẫu Sử Dụng

Room là một nhóm logic của các socket, cho phép server gửi sự kiện đến một nhóm mục tiêu cụ thể thay vì gửi cho tất cả mọi người.

Personal Room (`user-\${userId}`)



Mục đích: Gửi thông báo và dữ liệu cá nhân (lời mời kết bạn, tin nhắn riêng tư).

Vòng đời: Tự động tham gia khi kết nối, tự động rời khi ngắt kết nối.

Cách dùng:

```
io.to('user-userId').emit(...)
```

Workspace Room (`workspace-\${workspaceId}`)



Mục đích: Gửi các cập nhật trong một không gian làm việc chung (tạo task mới, thêm thành viên).

Vòng đời: Tham gia thủ công khi người dùng mở workspace, rời thủ công khi người dùng thoát ra.

Cách dùng: `io.to('workspace-workspaceId').emit(...)`

Document Room (`\${documentId}`)



Mục đích: Phục vụ cho tính năng chỉnh sửa tài liệu cộng tác thời gian thực.

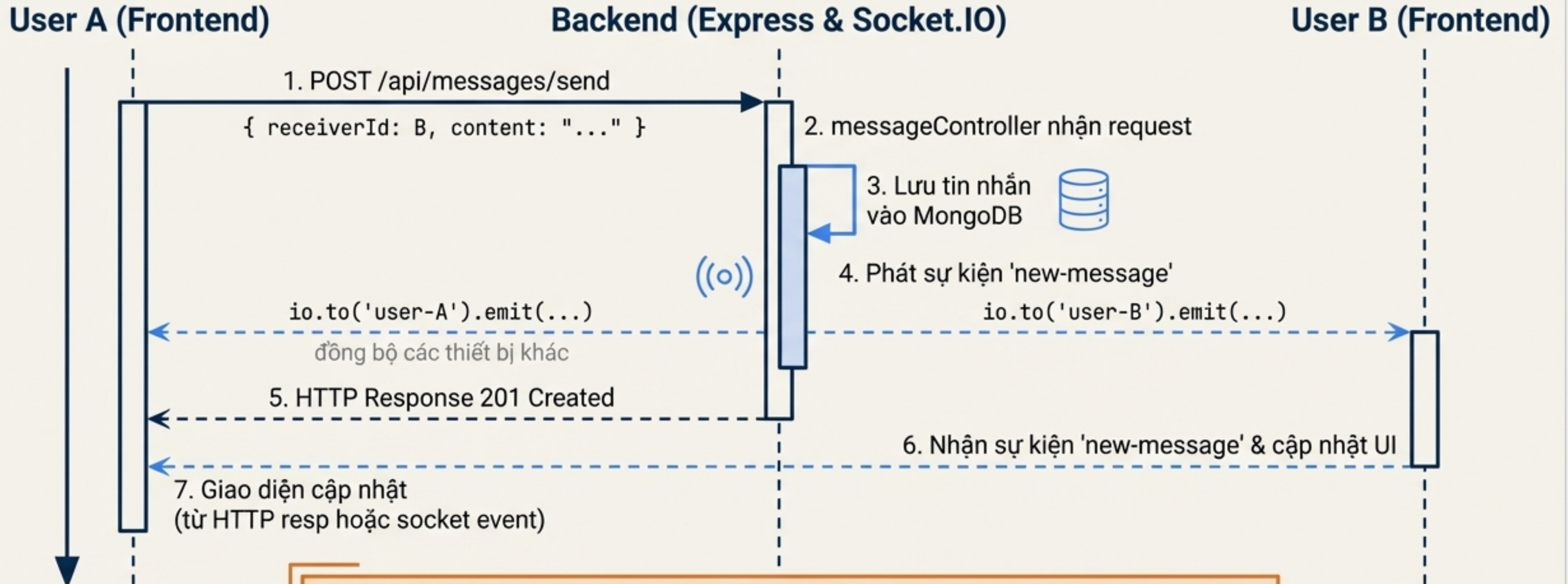
Vòng đời: Tham gia khi người dùng mở tài liệu, rời khi đóng lại.

Cách dùng:

```
socket.broadcast.to(documentId).emit(...)
```


Luồng Dữ Liệu: Gửi Tin Nhắn Giữa Hai Người Dùng

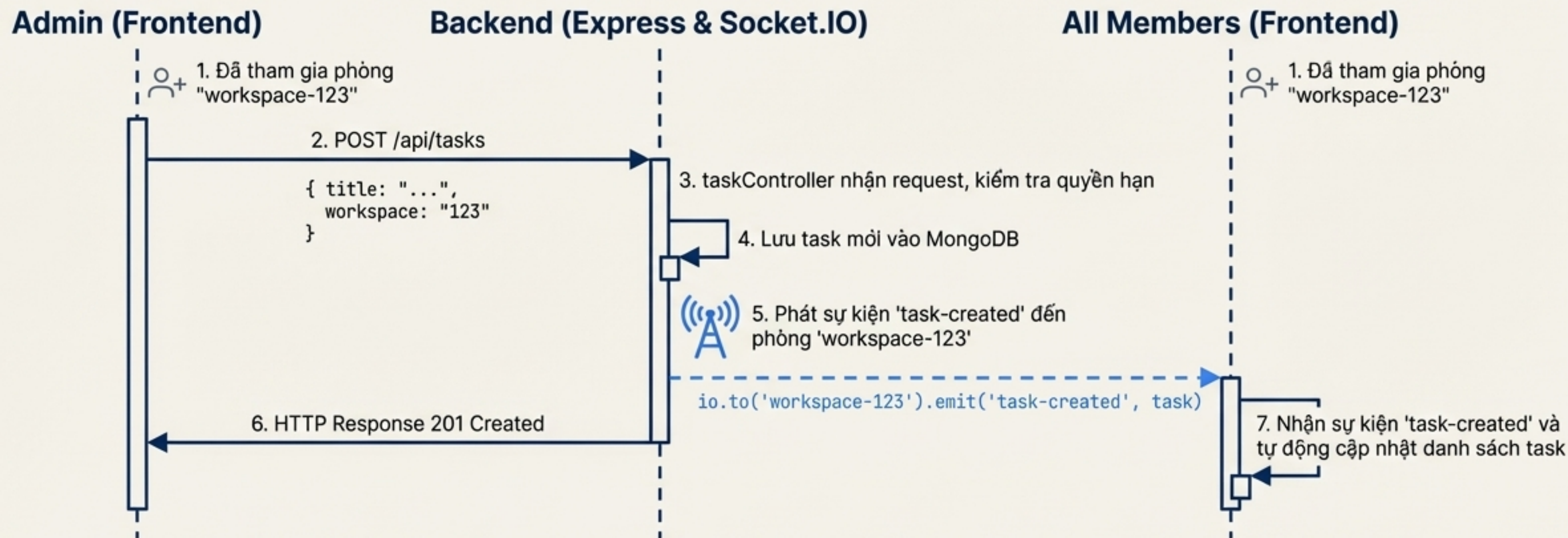
Người dùng A gửi tin nhắn cho người dùng B.



Luồng này kết hợp độ tin cậy của HTTP để lưu trữ dữ liệu và sức mạnh của **Socket.IO** để đồng bộ hóa giao diện người dùng ngay lập tức.

Luồng Dữ Liệu: Cập Nhật Task Trong Workspace

Scenario: Admin tạo một task mới trong một không gian làm việc. Tất cả thành viên trong workspace đó cần nhận được cập nhật.



Key Takeaway: Sử dụng room ``workspace-${workspaceId}`` đảm bảo rằng chỉ những người dùng đang hoạt động trong workspace liên quan mới nhận được cập nhật, giúp tiết kiệm băng thông và tài nguyên client.

Cẩm Nang Vận Hành: Testing và Debugging

Chiến Lược Testing

Backend (Unit Test với Jest)

- Sử dụng `socket.io-client` và `socket.io` server trong môi trường test.
- Kiểm tra việc phát và nhận sự kiện, đảm bảo payload chính xác.
- Mock các thành phần phụ thuộc như database.

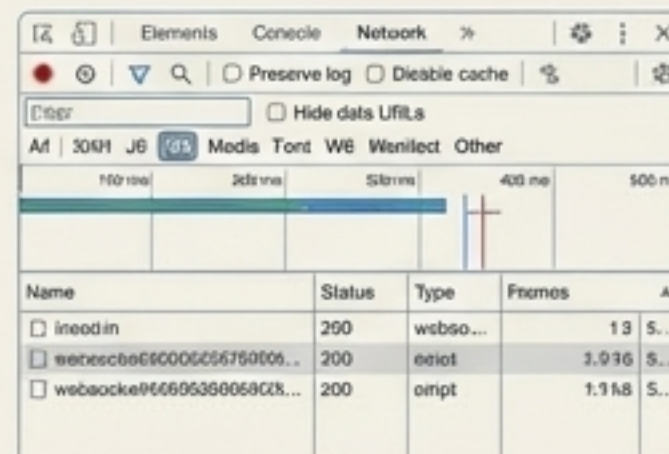
Frontend (Component Test với Jest & React Testing Library)

- Mock `socketService` để mô phỏng các sự kiện socket.
- Kiểm tra xem component có phản ứng đúng với các sự kiện được mô phỏng hay không (ví dụ: hiển thị tin nhắn mới).

Công Cụ Debugging Hữu Ích

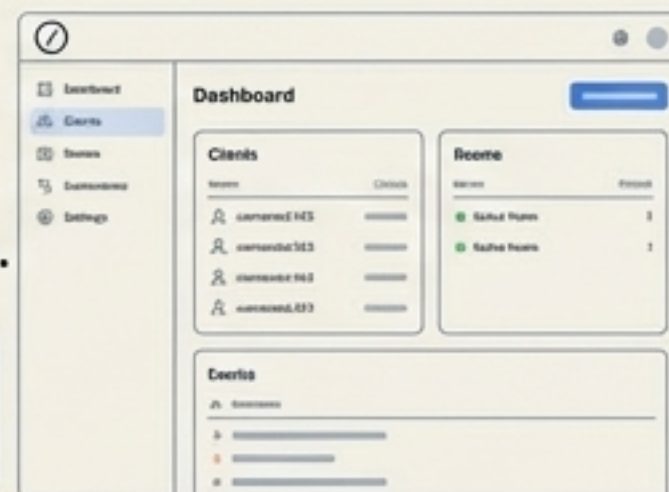
Chrome DevTools (Network > WS)

Theo dõi các frame WebSocket để xem dữ liệu được gửi và nhận trong thời gian thực.



Socket.IO Admin UI

Một giao diện web để theo dõi các socket đang kết nối, các phòng và các sự kiện đang diễn ra. Rất hữu ích để debug trực tiếp.



```
const { instrument } =  
require('@socket.io/admin-ui');
```

Logging Toàn Diện

Sử dụng `socket.onAny(...)` ở cả frontend và backend trong môi trường development để log tất cả các sự kiện, giúp dễ dàng theo dõi luồng dữ liệu.

Tham Chiếu: Danh Sách Toàn Bộ Sự Kiện

Connection & Room Events

connection	C → S	Kết nối với máy chủ socket
disconnect	C → S	Ngắt kết nối khỏi máy chủ socket
connect_error	S → C	Lỗi trong quá trình kết nối
join-workspace	C → S	Tham gia vào phòng của một không gian làm việc
leave-workspace	C → S	Rời khỏi phòng của một không gian làm việc
joined-workspace	S → C	Xác nhận đã tham gia phòng thành công
join-document	C → S	Tham gia vào phòng của một tài liệu
leave-document	C → S	Rời khỏi phòng của một tài liệu

Message & Typing Events

new-message	S → C	Thông báo có tin nhắn mới
message-read	S → C	Thông báo tin nhắn đã được đọc
message-deleted	S → C	Thông báo tin nhắn đã bị xóa
typing	C → S	Báo hiệu người dùng đang gõ phím
stop-typing	C → S	Báo hiệu người dùng đã ngừng gõ phím
user-typing	S → C	Chuyển tiếp sự kiện 'typing' đến người nhận
user-stop-typing	S → C	Chuyển tiếp sự kiện 'stop-typing' đến người nhận

Friend Connection Events

friend-request-received	S → C	Nhận được một lời mời kết bạn mới
friend-request-accepted	S → C	Lời mời kết bạn đã được chấp nhận
friend-removed	S → C	Một người bạn đã bị xóa khỏi danh sách

Workspace & Document Events

task-created	S → C	Thông báo một nhiệm vụ mới được tạo
task-updated	S → C	Thông báo một nhiệm vụ đã được cập nhật
task-deleted	S → C	Thông báo một nhiệm vụ đã bị xóa
member-added	S → C	Thông báo một thành viên mới được thêm vào
member-removed	S → C	Thông báo một thành viên đã bị xóa
send-changes	C → S	Gửi các thay đổi của tài liệu lên máy chủ
receive-changes	S → C	Nhận các thay đổi của tài liệu từ máy chủ
new-comment	S → C	Thông báo có bình luận mới trong tài liệu

Error Events

error	S → C	Dùng cho các lỗi xác thực hoặc nghiệp vụ.
--------------	--------------	---

Tối Ưu Hóa Hiệu Năng

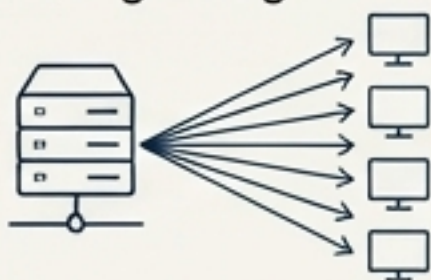
Viết code Socket.IO không chỉ để chạy đúng, mà còn để chạy nhanh và hiệu quả.

1. Nhắm Mục Tiêu Bằng Room (Không Broadcast)

❌ **Don't**

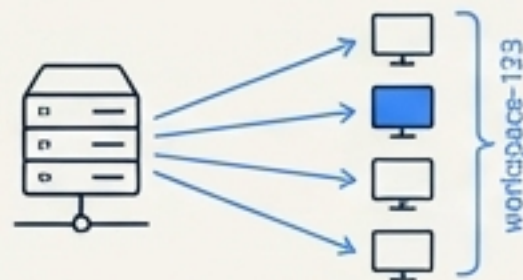
`io.emit('event', data)`
(Gửi cho TẤT CẢ mọi người)

Giới hạn phạm vi phát sóng sự kiện để giảm tải máy chủ và tiết kiệm băng thông.



✅ **Do**

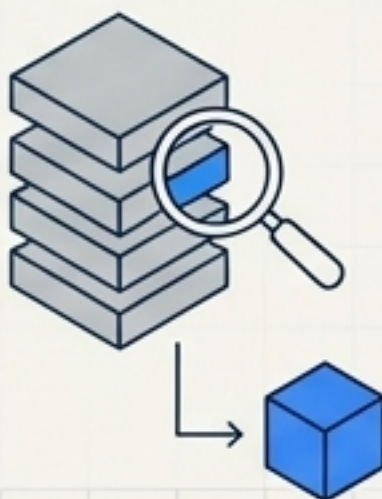
`io.to('workspace-123').emit('event', data)`
(Chỉ gửi cho những ai cần)



2. Tối Ưu Hóa Payload

Chỉ gửi những dữ liệu đã thay đổi, không gửi toàn bộ object.

Ví dụ:
Instead of
``entireUserObject``, send
`{ userId, changes:
{ avatar: newUrl } }`



3. Dọn Dẹp Listener (Cleanup)

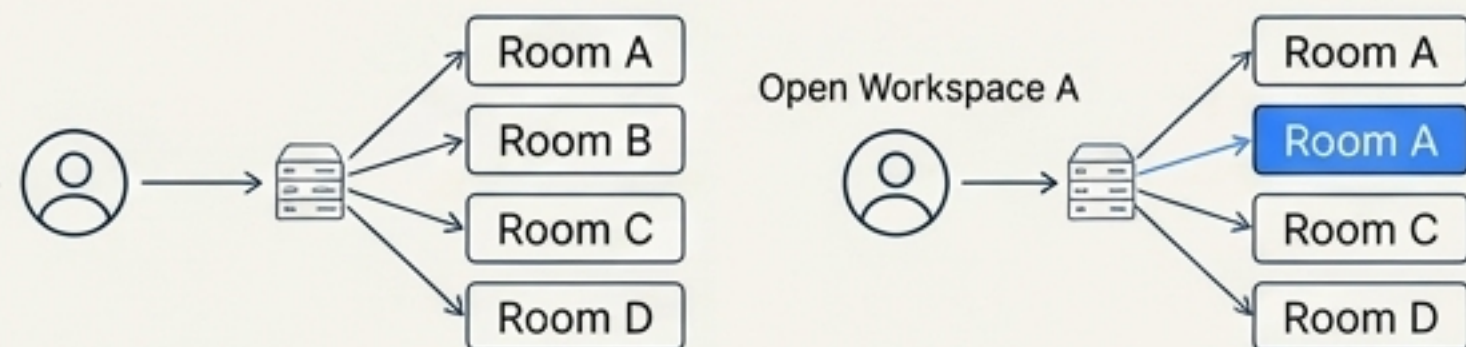
Luôn sử dụng hàm cleanup trong ``useEffect`` để gọi ``socket.off()``. Đây là nguyên nhân số một gây rò rỉ bộ nhớ ở frontend.

```
useEffect(() => {  
  socket.on('event', handler);  
  return () => socket.off('event', handler);  
}, []);
```



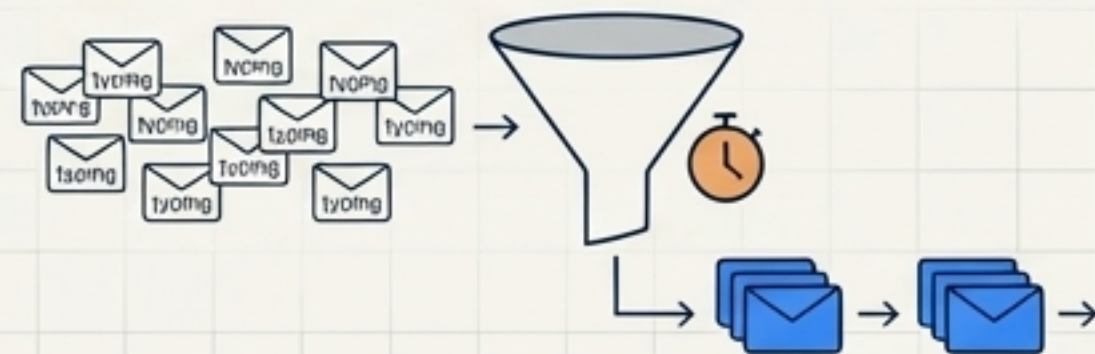
4. Tham Gia Room Theo Nhu Cầu (Lazy Joining)

Không ``join`` tất cả các room của người dùng ngay khi kết nối. Chỉ ``join`` khi người dùng thực sự truy cập vào chức năng đó (ví dụ: mở một workspace).



5. Gộp Sự Kiện (Event Batching/Debouncing)

Đối với các sự kiện tần suất cao (như gõ phím, di chuyển chuột), hãy gộp chúng lại và **gửi đi định kỳ (ví dụ: mỗi 100ms)** thay vì gửi liên tục.



Các Quy Tắc Vàng: Nên và Không Nên

✓ NÊN LÀM (DO)

1. **Luôn dọn dẹp listener:** ``return () => socket.off(...)`` trong ``useEffect``.
2. **Sử dụng room để nhắm mục tiêu:** `io.to(roomName).emit(...)``.
3. **Xác thực dữ liệu từ client:** Luôn kiểm tra và làm sạch mọi payload nhận được từ client trước khi xử lý.
4. **Xử lý kết nối lại:** Chuẩn bị kịch bản client bị mất kết nối và kết nối lại (re-join room, đồng bộ lại trạng thái).
5. **Luôn xác thực socket:** Dùng middleware để đảm bảo mọi kết nối đều được định danh và có quyền hợp lệ.

✗ KHÔNG NÊN LÀM (DON'T)

1. **Broadcast cho tất cả:** `io.emit(...)` là một anti-pattern trong hầu hết các trường hợp.
2. **Quên dọn dẹp listener:** Gây ra memory leak và các hành vi không mong muốn.
3. **Gửi payload lớn, không cần thiết:** Gây lãng phí băng thông và làm chậm client.
4. **Tin tưởng tuyệt đối dữ liệu client:** Mọi dữ liệu từ client đều có thể bị giả mạo.
5. **Tạo nhiều kết nối socket:** Sử dụng mô hình Singleton Service để quản lý một kết nối duy nhất.

Xử Lý Lỗi và Đảm Bảo Hoạt Động Ổn Định

1. Lỗi Kết Nối

Phía Frontend

- Lắng nghe sự kiện `connect_error`.

Logic

- Nếu lỗi là `Invalid token` hoặc `Token expired`, hãy thử refresh access token và kết nối lại.

Phía Backend

- Middleware `socketAuth` trả về lỗi rõ ràng: `next(new Error('Invalid token'))`.

2. Lỗi Sự Kiện (Event Errors)

Phía Backend

- Xác thực payload của mỗi sự kiện.
- Kiểm tra quyền hạn của người dùng (`socket.userId`) trước khi thực hiện hành động.
- Nếu có lỗi, phát sự kiện `error` về cho client: `socket.emit('error', { message: '...' })`.

Phía Frontend

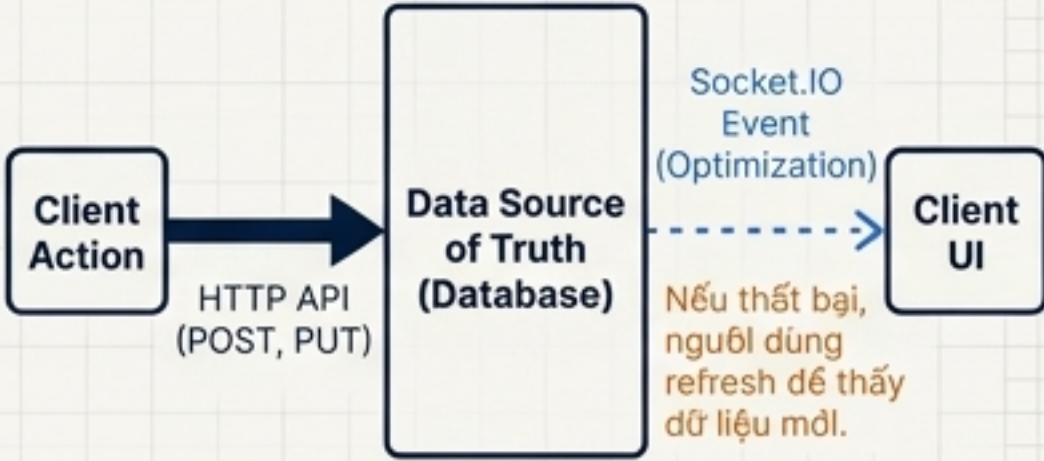
- Đăng ký một listener toàn cục cho sự kiện `error` để hiển thị thông báo cho người dùng (ví dụ: Toast notification).

3. Nguyên Tắc Graceful Degradation

Triết lý

Hệ thống vẫn phải hoạt động được ngay cả khi socket không kết nối.

Kiến trúc



Tổng Kết và Hướng Phát Triển Tương Lai

Các Nguyên Tắc Kiến Trúc Cốt Lõi

- ✓ **Giao tiếp hai chiều, thời gian thực:** Nền tảng cho các tính năng cộng tác.
- ✓ **Xác thực an toàn với JWT:** Mọi kết nối đều được bảo vệ ngay từ handshake.
- ✓ **Nhằm mục tiêu hiệu quả bằng Room:** Giảm thiểu dữ liệu không cần thiết và tăng khả năng mở rộng.
- ✓ **Tự động kết nối lại và xử lý lỗi:** Đảm bảo trải nghiệm người dùng liền mạch.
- ✓ **Kiến trúc tách biệt:** Logic nghiệp vụ nằm trong controller, socket chỉ để đồng bộ.

Lộ Trình Phát Triển (Next Steps)



Mở rộng quy mô (Scaling): Tích hợp Redis Adapter để hỗ trợ nhiều server instance (horizontal scaling).



Bảo mật nâng cao: Triển khai Rate Limiting trên mỗi socket để chống spam.



Giám sát (Monitoring): Theo dõi các chỉ số quan trọng (số lượng kết nối, số sự kiện/giây).



Mã hóa đầu cuối: Cân nhắc mã hóa cho các dữ liệu nhạy cảm.